DAA LAB 9

Kshitij Kejriwal-252010005

1st April 2026

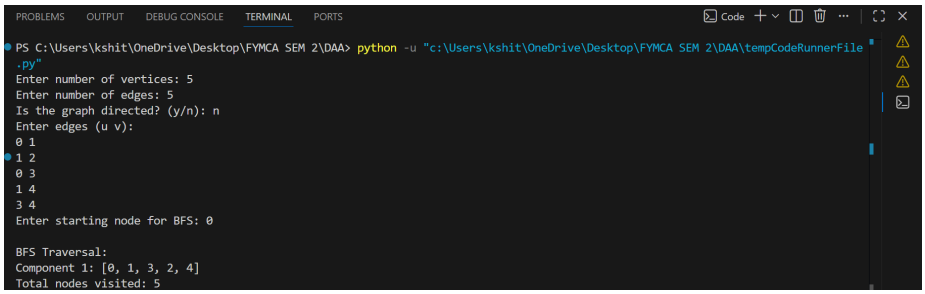
Breadth First Search (BFS)

1 Code:

```python
from collections import deque

class Graph:
    def __init__(self, vertices, directed=False):
        self.V = vertices
        self.graph = {i: [] for i in range(vertices)}
        self.directed = directed

    def add_edge(self, u, v):
        self.graph[u].append(v)
        if not self.directed:
            self.graph[v].append(u)

    def bfs(self, start, visited):
        queue = deque()
        queue.append(start)
        visited[start] = True

        traversal = []

        while queue:
            node = queue.popleft()
            traversal.append(node)

            for neighbor in self.graph[node]:
                if not visited[neighbor]:
                    visited[neighbor] = True
                    queue.append(neighbor)

        return traversal

    def bfs_full(self, start):
```

```python
        visited = [False] * self.V
        all_components = []
        total_nodes = 0

        # BFS from starting node
        if not visited[start]:
            comp = self.bfs(start, visited)
            all_components.append(comp)
            total_nodes += len(comp)

        # Handle disconnected components
        for i in range(self.V):
            if not visited[i]:
                comp = self.bfs(i, visited)
                all_components.append(comp)
                total_nodes += len(comp)

        return all_components, total_nodes


# -------- USER INPUT --------
n = int(input("Enter number of vertices: "))
m = int(input("Enter number of edges: "))

type_graph = input("Is the graph directed? (y/n): ")
directed = True if type_graph.lower() == 'y' else False

g = Graph(n, directed)

print("Enter edges (u v):")
for _ in range(m):
    u, v = map(int, input().split())
    g.add_edge(u, v)

start = int(input("Enter starting node for BFS: "))

# -------- BFS --------
components, total = g.bfs_full(start)

# -------- OUTPUT --------
print("\nBFS Traversal:")

for idx, comp in enumerate(components):
    print(f"Component {idx + 1}: {comp}")

print("Total nodes visited:", total)
```

2 Output:



```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Code

PS C:\Users\kshit\OneDrive\Desktop\FYMCA SEM 2\DAA> python -u "c:\Users\kshit\OneDrive\Desktop\FYMCA SEM 2\DAA\tempCodeRunnerFile
.py"
Enter number of vertices: 5
Enter number of edges: 5
Is the graph directed? (y/n): n
Enter edges (u v):
0 1
1 2
0 3
1 4
3 4
Enter starting node for BFS: 0

BFS Traversal:
Component 1: [0, 1, 3, 2, 4]
Total nodes visited: 5
```

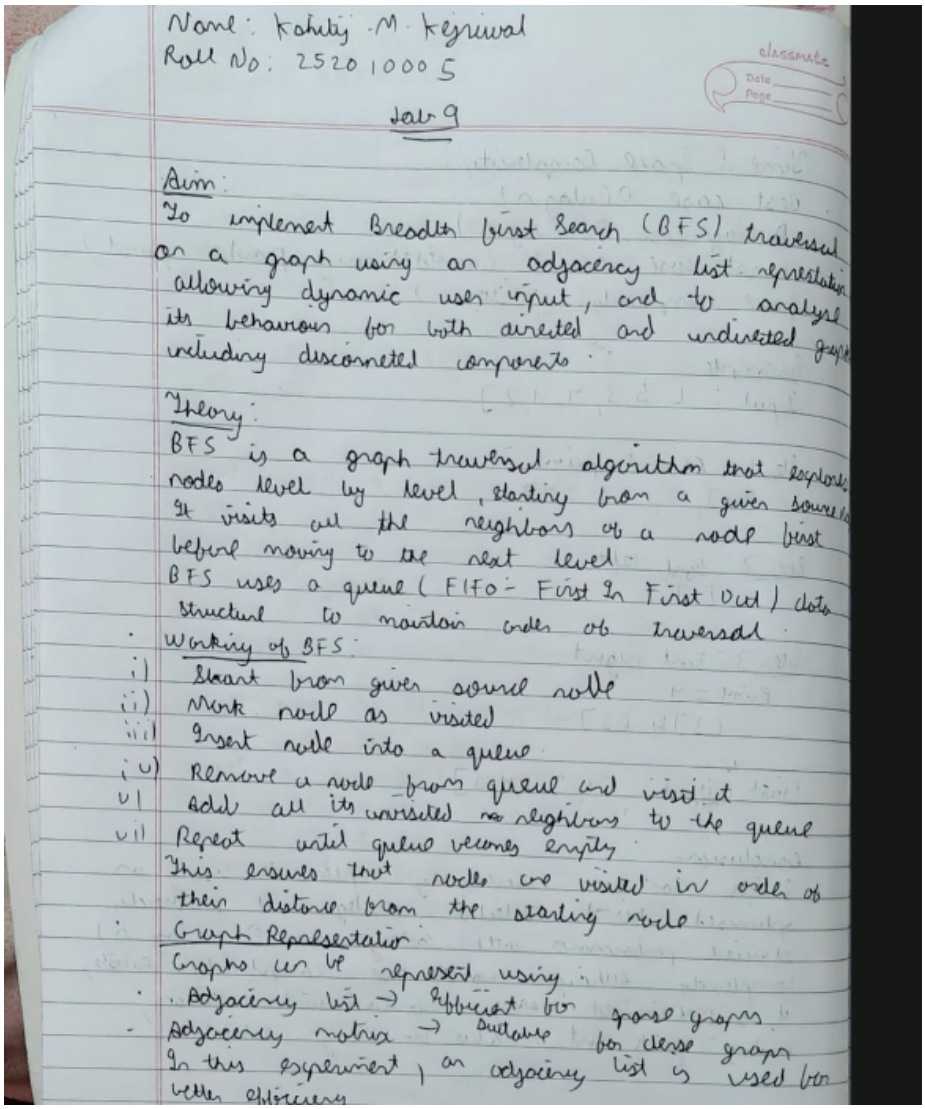
Figure 1: Output

3 Pseudocode:

Algorithm 1 Breadth First Search (BFS)

```
procedure BFS(G, start)
    Create queue Q
    Mark start as visited
    Enqueue start into Q
    while Q is not empty do
        node ← Dequeue(Q)
        Print node
        for each neighbor of node do
            if neighbor not visited then
                Mark neighbor visited
                Enqueue neighbor
            end if
        end for
    end while
end procedure
```

4 Explanation:



Name: Kshitij M Kejriwal
Roll No: 2520100005

Lab 9

Aim:
To implement Breadth first Search (BFS) traversal on a graph using an adjacency list representation allowing dynamic user input, and to analyze its behaviour for both directed and undirected graphs including disconnected components.

Theory:
BFS is a graph traversal algorithm that explores nodes level by level, starting from a given source node. It visits all the neighbors of a node first before moving to the next level.
BFS uses a queue (FIFO - First In First Out) data structure to maintain order of traversal.

Working of BFS:
i) Start from given source node
ii) Mark node as visited
iii) Insert node into a queue
iv) Remove a node from queue and visit it
v) Add all its unvisited neighbors to the queue
vi) Repeat until queue becomes empty
This ensures that nodes are visited in order of their distance from the starting node.

Graph Representation:
Graphs can be represent using:
- Adjacency list → Efficient for sparse graphs
- Adjacency matrix → Suitable for dense graphs
In this experiment, an adjacency list is used for better efficiency

Figure 2: Explanation

- Handling Directed and Undirected graphs:
  - In undirected graph, edges are bidirectional
  - In directed graph, edges have specific direction.
  - The BFS works for both by modifying how edges are added.
- Disconnected graph:
  - If graph is disconnected, BFS from 1 node will not visit all nodes
  - To handle this:
  - BFS is applied to all unvisited nodes.
  - Each traversal represents a connected component.

Time and Space Complexity:

- Time Complexity = $O(V+E)$
  (V = vertices, E = edges)
- Space complexity = $O(V)$ (for queue and visited array)

- Application of BFS:
  - i) Finding shortest path in unweighted graphs
  - ii) Checking connectivity of graph
  - iii) Finding connected components.
  - iv) Used in network broadcasting and routing

Advantages:

- i) Simple and easy to implement
- ii) Guarantees shortest path in unweighted graphs
- iii) Efficient for large graphs

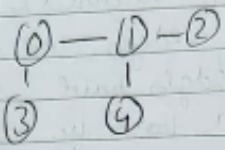
Limitations:

- i) Require extra memory for queue
- ii) Not suitable for very deep graphs compared to DFS

Figure 3: Explanation

Example:

Graph:



Adjacency list:

0 -> 1, 3

1 - 0, 2, 4

2 - 1

3 - 0

4 - 1

Starting node = 0

Step 1: Start at 0

queue = [0]

visited = 0

Step 2: visit 0 Add neighbors (1, 3)

queue = [1, 3]

visited = {0, 1, 3}

Step 3: visit 1 add neighbors (2, 4)

queue = [3, 2, 4]

visited = {0, 1, 2, 3, 4}

Step 4: visit 3 no new nodes

queue = [2, 4]

Step 5: visit 2 -> no new nodes

queue = [4]

Step 6: visit 4 -> no new nodes

queue = [ ]

Final BFS traversal = [0, 1, 3, 2, 4]

Conclusion:

BFS traversal was successfully implemented using a queue. It efficiently explores all nodes level-wise and handles disconnected graphs

Figure 4: Explanation